

La preuve programme

-

Le plus haut standard d'
écriture de logiciel

Les garanties sur un logiciel

- Certains domaines d'application de la programmation ont besoin de garanties sur les programmes sur lesquels ils dépendent
- Exemples :
 - Systèmes embarqués de contrôle commande de véhicules (trams, trains, avions)
 - Systèmes de communication de satellite (compliqué à aller reflasher !)
 - Electronique médicale (flemme que ton pacemaker segfault)
 - Systèmes bancaires

Quels types de garanties sont désirables ?

- “Pas de bug”, c’est le but global, mais en pratique on parle de :
 - Garantie de terminaison : si le hardware fonctionne et qu’on lance le programme/une partie du programme, cela **VA** se terminer (bonus si on peut garantir un temps sur cette exécution)
 - Spécifications : le client veut un programme qui fasse A,B,C et D, on veut pouvoir lui prouver qu’on a fait A,B,C,D et pas A,D,G,E
 - Garantie d’état : SI le hardware vient à défaillir, comment est-ce que le programme peut reprendre sans faute dès que le hardware reprend ?

Une première approche : le test (au sens critique)

- Exemple :
 - Ma fonction prend un entier 8 bit, renvoie un entier 8 bit
 - Idée : on va faire un programme qui appelle ma fonction sur tous les entiers 8 bits, et vérifier la cohérence des résultats en sortie !
 - Sur 256 entrées et sorties possibles, la vérification pourrait même se faire à la main
- Ça marche, et c'est l'approche la plus populaire pour faire du logiciel critique relativement sûr

Les limites des tests : pourquoi ?

- Exemple :
 - Ma fonction prend en entrée 2 entiers 64 bits
 - Taille de l'espace de test brut : 2^{128}
 - Même avec une fonction de vérification qui s'exécute en 1 cycle CPU, sur 1 CPU milieu de gamme moderne, on a ~1 milliard de milliard d'années de temps de vérification
- Comment est-ce qu'on fait des logiciels réels avec alors ?

Les limites des tests : les raccourcis et les contraintes

- La solution à l'explosion des espaces de test est d'en explorer le moins possible, avec
 - Des contraintes : par exemple ma fonction prend des entiers 64 bits en entrée, mais n'en utilise que 42 bits par entier
 - Des raccourcis : si ma fonction fait une simple multiplication avec une opération simple en plus, on “sait” qu'on a pas à tester autant les cas qui utilisent les nombres représentables sur 1 à 31 bits, car leur multiplication sera nécessairement représentable sur 64 bits
 - Symmétries : on “sait” que $a + b == b + a$ avec des entiers, donc pas besoin de tester la moitié des cas !

Les tests en pratique

- En réalité, on va donc utiliser plein de raccourcis plus ou moins corrects pour réduire l'espace de test, imposer des contraintes sur le programme...
- Il faut vérifier ces contraintes et ces raccourcis, en théorie, on peut réduire l'espace de recherche en ne perdant rien en sûreté !
- En pratique...
 - <https://time.com/archive/6921883/a-330-million-case-of-jet-lag/>
 - Les F22 n'utilisent que du code testé, et leur système de navigation panique en passant entre les 2 fuseaux horaires les plus éloignés sur l'océan Pacifique ! Un raccourci de trop...
- Même avec toutes les méthodes de test efficaces modernes, on a un facteur $\geq x80$ de coût de développement par rapport au programme moyen

L'intuition

- Tous ces raccourcis, ces contraintes... on “sait” qu'ils sont corrects
(jusqu'à ce qu'ils soient pas corrects)
- Et si on le **prouvait** pour en être sûr ?

LA PREUVE PROGRAMME !!!!!

L'approche “mathématique” : la preuve programme

- Au lieu de tester le possible, on établit la relation mathématique **formelle** entre l'input et l'output (spécification) et les contraintes **formelles** du programme
- On associe à la grammaire du langage utilisé une signification mathématique, puis on prouve que le programme $\Rightarrow$ la spécification

Exemple “trivial” : la fonction de somme + 1

- On veut une fonction qui :
 - Prend 2 entiers positifs $\leq 40\,000$ en entier
 - Renvoie la somme de ces 2 entiers, + 1
 - On parle de somme au sens mathématique, donc la somme de 2 entiers positifs doit naturellement AU MOINS être supérieure ou égale aux 2 entiers

La preuve programme en général

- Le concept s'étend sur les boucles avec des preuves par récursion
- Avec des fonctions, des opérations simples et des boucles, on peut exprimer plein de programmes réels !
- Comme la preuve est forte, on **sait** qu'on a pas de zone d'ombre ou de raccourcis douteux !

Les limites théoriques de la preuve programme

- La machine de Turing : il existe des programmes qu'on ne peut pas prouver entièrement
- Les systèmes réels peuvent être théoriquement très complexes :
 - Le modèle mémoire : c'est quoi mathématiquement, **formellement** une erreur OOM ? Comment la gérer ?
 - Sur un OS moderne de bureau : tout simplement impossible de faire une preuve de terminaison sans contraintes fausses (le scheduling casse tout, même en mono-coeur, mono-thread)
 - Faut définir et prouver formellement tout l'assembleur final pour que la preuve soit utile, pas facile !

Les limites pratiques de la preuve programme

- C'est extrêmement complexe : le test est déjà compliqué mais requiert relativement peu d'expertise, mais pour la preuve -> expertise mathématique **requis**
- Faut complètement spécifier le programme, ça va nécessairement prendre plus de temps que tout autre méthode à la main
- Ce qui est théoriquement difficile sur certains systèmes réels devient économiquement irréaliste
- **Des millions de lignes de codes critiques sont écrites chaque année, comment les rendre sûres ?**

Les assistants de preuve

Les assistants de preuve : l'idée

- La preuve est souvent intellectuellement intensive, mais majoritairement répétitive (à un certain niveau d'abstraction)
- Les ordinateurs sont parfaitement capable de faire des maths formelles, si on les code pour
- Tout ce qu'il faut :
 - Un langage avec une grammaire mathématique assez développée pour exprimer en même temps le programme et ses contraintes
 - Un programme magique (lui-même prouvé) qui peut faire une preuve à partir d'un problème énoncé dans ce langage magique

Facile ! (pas facile (très difficile))

Les assistants de preuve : la pratique

- Les assistants de preuve modernes sont utilisés autant pour les mathématiques que pour la programmation
- Si la preuve compile, c'est qu'elle est **correcte** , de manière forte !
- Exemple : Lean
 - <https://lean-lang.org/>

Les assistants de preuve : les cas d'usage réels

- Compilateur C prouvé : CompCert (<https://compcert.org/>) prouvé avec Rocq (<https://rocq-prover.org/>)
 - Pourquoi un compilateur prouvé ? demandez à AMD :
<https://github.com/ROCm/ROCm/issues/5826>
- Les systèmes logiciels d'aiguillages automatiques de tram en France
- AlphaProof
<https://deepmind.google/blog/ai-solves-imo-problems-at-silver-medal-level/>

Les assistants de preuve : les limites

- Il faut toujours formaliser le programme à prouver
- Il faut toujours avoir une certaine expertise mathématique pour les utiliser efficacement
- Turing-complétude : la preuve peut prendre un temps (pratiquement ou littéralement) infini, ou être tout simplement **impossible** avec les outils actuels
- La définition complète de systèmes modernes complexes est toujours un sujet de recherche actif, les assistants de preuve sont toujours majoritairement limités aux systèmes embarqués !

Fin !

Crédits

Slides : Pierre Weisse